

Skillab

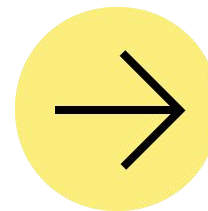
AI Agent Development

Richard Șerban Francu

Senior AI Engineer @UCE Systems



Programa cursului

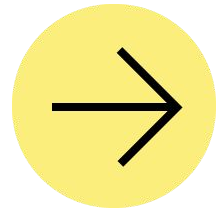


1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. **Procesarea și extragerea documentelor**
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF

Lecția 03

Procesarea și extragerea documentelor

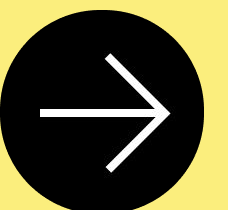
- Încărcarea și procesarea diferitelor tipuri de documente (PDF, DOCX, TXT, CSV)
- Strategii de fragmentare (chunking) pentru diferite cazuri de utilizare
- Extragerea datelor structurate folosind modele Pydantic



Procesarea și extragerea documentelor

Document Loaders

Cum citim documente și le pregătim pentru AI



Ce Sunt Document Loaders?



factura.pdf

PDF



contract.docx

DOCX



export.csv

CSV



termeni.txt

TXT



Document Loader

Abstracție care parsează orice format și returnează o structură uniformă.

```
1 from langchain_community.document_loaders import PyPDFLoader
2
3 # Încarcă PDF-ul și parsează conținutul
4 loader = PyPDFLoader("factura.pdf")
5 documents = loader.load()
6
7 # Fiecare pagină devine un Document
8 doc = documents[0]
9
10 print(doc.page_content) # textul paginii
11 print(doc.metadata)    # {'source': 'factura.pdf', 'page': 0}
12
```



Analogie: adapter de priză — formatul diferă, textul e uniform

De Ce Loaders Diferiți?

Format	Structură	Provocări
 PDF	Layout complex, fonturi, imagini	OCR, tabele, multi-pagină
 DOCX	XML comprimat, stiluri, metadata	Formatare, footnotes, comentarii
 TXT	Text simplu	Encoding (UTF-8, cp1250...)
 CSV	Date tabulare, delimiters	Headers, separatori diferiți

LangChain Document Loaders



```
1 from langchain_community.document_loaders import (  
2     PyPDFLoader,      # PDF  
3     Docx2txtLoader,   # Word  
4     TextLoader,       # TXT  
5     CSVLoader         # CSV  
6 )  
7
```



Testate și menținute de comunitate



API uniform — toate au .load()



Gestionează edge cases automat



Returnează același tip: Document



Nu face asta — manual parsing:



```
1 # ❌ 200+ linii manual parsing  
2 # ❌ Encoding bugs (cp1250 vs UTF-8)  
3 # ❌ Crash pe PDF-uri corupte
```

Structura Document

```
from langchain_core.documents import Document

doc = Document(
    page_content="Text extras...",
    metadata={
        "source": "factura_001.pdf",
        "page": 0
        # poti adauga custom metadata
    }
)
```

page_content

(str)

Textul extras din document

metadata

(dict)

Informații despre sursă
(fișier, pagină, etc.)

De ce metadata contează:

■ Tracking — știi de unde vine textul

■ Debugging — ce pagină a creat problema

■ RAG — citarea surselor (Lecția 4)

PyPDFLoader PDF

Cel mai frecvent

Format ideal pentru facturi și contracte scanabile.



```
1  from langchain_community.document_loaders import PyPDFLoader
2
3  loader = PyPDFLoader("facturi/factura_001.pdf")
4  documents = loader.load()
5
6  # 1 Document per pagină PDF
7  print(f"Pagini extrase: {len(documents)}")
8
9  # Iterăm prin fiecare pagină
10 for doc in documents:
11     page_num = doc.metadata["page"]
12     preview = doc.page_content[:80]
13     print(f"\n--- Pagina {page_num} ---")
14     print(preview)
15
```

Caracteristici

- ✓ 1 Document per pagină PDF
- ✓ Metadata include nr. paginii
- ✓ Funcționează cu PDF searchable

Limitări

- ✗ PDF scanate → nevoie de OCR
- ✗ Tabele complexe → structura se pierde
- ✗ Imagini → nu sunt extrase

Docx2txtLoader — Word

Pentru contracte, oferte, documente editate frecvent.

```
1 from langchain_community.document_loaders import Docx2txtLoader
2
3 loader = Docx2txtLoader("contract_001.docx")
4 documents = loader.load()
5
6 # 1 Document pentru tot fișierul (nu împarte pe pagini)
7 print(f"Documents: {len(documents)}") # 1
8
9 doc = documents[0]
10 print(doc.metadata) # {'source': 'contract_001.docx'}
11 print(doc.page_content[:200]) # primele 200 caractere
12
```

Use case: Analiză bulk · Extragere clauze · Comparare versiuni

Output

1 Document — tot fișierul
(nu împarte pe pagini)

Ce extrage

Doar text
(nu formatare, culori, comentarii)

Când folosești

Contracte · Oferte · Documente Word

TextLoader & CSVLoader



TextLoader



```
1 # TextLoader – encoding contează pentru diacritice
2 text_loader = TextLoader("termeni.txt", encoding="utf-8")
3 text_docs = text_loader.load()
4 print(f"Text: {len(text_docs)} Document") # 1 – tot fișierul
5
```

⚠ Encoding matters:

utf-8 → Română, diacritice
cp1250 → Windows Romanian vechi



CSVLoader



```
1 # CSVLoader – 1 Document per rând
2 csv_loader = CSVLoader("facturi_export.csv")
3 csv_docs = csv_loader.load()
4 print(f"CSV: {len(csv_docs)} Documents") # ex: 50 rânduri
5
```

Output per rând:

numar: 2024-001
data: 2024-02-15
suma: 6000 RON

Registry Pattern



```
1  # Registry: extensie → loader potrivit
2  LOADER_REGISTRY = {
3      "pdf": PyPDFLoader,
4      "docx": Docx2txtLoader,
5      "txt": TextLoader,
6      "csv": CSVLoader,
7  }
8
9  def load_document(path: str):
10     ext = Path(path).suffix.lstrip(".").lower()
11     loader_cls = LOADER_REGISTRY[ext]
12     return loader_cls(path).load()
```

Beneficii →

Un singur entry point

→ `load_document("orice.pdf")`

Extensibil: format nou = 1 linie in dict

Refolosit in pipeline (S4) si agent (S5)

Error Handling



```
1 def load_document(path: str):
2     ext = Path(path).suffix.lstrip(".").lower()
3     if ext not in LOADER_REGISTRY:
4         raise ValueError(f"Tip nesuportat: .{ext}")
5     return LOADER_REGISTRY[ext](path).load()
6
7 try:
8     docs = load_document("factura.xlsx")
9 except FileNotFoundError:
10     print("Fisier inexistent")
11 except ValueError as e:
12     print(e)                # Tip nesuportat: .xlsx
13 except UnicodeDecodeError:
14     print("Encoding gresit")
15
```

FileNotFoundError

Fisier inexistent
sau calea e gresita

ValueError

Format nesuportat
(.xlsx, .pptx etc.)

UnicodeDecodeError

Encoding gresit (TXT)
Specifica encoding="utf-8"

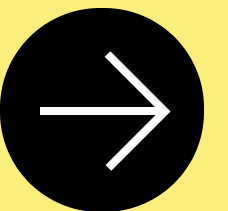
Sumar — Document Loaders

Loader	Format	Utilizare	Output
 PyPDFLoader	PDF	Facturi, contracte	1 doc/pagină
 Docx2txtLoader	DOCX	Contracte Word	1 doc/fișier
 TextLoader	TXT	Termeni, docs simple	1 doc/fișier
 CSVLoader	CSV	Export date	1 doc/rând

Instalare: `pip install langchain-community pypdf docx2txt` | **Next → Secțiunea 2: Text Splitters**

Text Splitters și Chunking

Cum împărțim documente mari pentru procesare AI



De Ce Chunking?

Problema

Contract 100 pagini = ~50,000 tokens
Nu încape în majoritatea modelelor!

Soluția: CHUNKING

Împarte documentul în bucăți
(chunks) mai mici, procesabile independent.

Model	Context	Cost
GPT-3.5	~4K tokens	leftin
GPT-4	~128K tokens	Scump
Claude 3	~200K tokens	Foarte scump
Llama 3.2	~128K tokens	Local (RAM)

Matematică rapidă:

1 token ≈ 4 caractere

1 pagină ≈ 500 tokens

100 pagini = ~50,000 tokens

→ Nu încape în GPT-3.5 / GPT-4!

Ce Este Chunking?

Document Original (10,000 caractere)

CONTRACT DE PRESTĂRI SERVICII ... 100 pagini ... SEMNĂTURI



Chunk 1

1,000 caractere

- ✓ Sens independent
- ✓ Încapă în model

Chunk 2

1,000 caractere

- ✓ Sens independent
- ✓ Încapă în model

Chunk 3

1,000 caractere

- ✓ Sens independent
- ✓ Încapă în model

Chunk 4

1,000 caractere

- ✓ Sens independent
- ✓ Încapă în model

Fiecare chunk: procesat separat • are sens independent • se potrivește în context window

Parametrii Cheie

chunk_size

Număr de caractere per chunk

```
chunk_size=1000
```

- ✗ Prea mic (100) → context fragmentat
- ✓ Sweet spot: 500–2,000 caractere
- ⚠ Prea mare (10,000) → depășești limite

chunk_overlap

Caractere comune între chunks consecutive

```
chunk_overlap=200 # 20%
```

Previne pierderea contextului la granițe
Formulă: $\text{overlap} = \text{chunk_size} \times 10\text{--}20\%$

Overlap în practică:

✗ Fără overlap
Chunk 1: "...conform"
Chunk 2: "specificațiilor..." ← context rupt!

✓ Cu overlap
Chunk 1: "...conform specificațiilor"
Chunk 2: "conform specificațiilor anexate..."

RecursiveCharacterTextSplitter

Cel mai folosit

```
1 splitter = RecursiveCharacterTextSplitter(  
2     chunk_size=1000,  
3     chunk_overlap=200,  
4     # Ordine: paragrafe → linii → cuvinte → caractere  
5     separators=["\n\n", "\n", " ", ""],  
6 )  
7  
8 contract = (  
9     "Art. 1 - Obiectul\n\nPrestatorul se obligă...\n\n"  
10    "Art. 2 - Durata\n\nContractul intră în vigoare...\n\n"  
11    "Art. 3 - Plata\n\nBeneficiarul achită..."  
12 )
```

Cum funcționează:

1 \n\n Paragrafe (ideal)

2 \n Linii

3 Cuvinte

De ce "Recursive"?

Încearcă separatori în ordine — de la cei mai naturali la cei mai mecanici.
Păstrează structura semantică a textului → chunks cu sens intact.

```
# Art. 1 - Obiectul\n\nPrestatorul se obligă...\n\nArt. 2 - Durata  
→ Split la \n\n → 3 chunks la articole (semantic!)
```

Comparație Strategii

Splitter	Împarte la	Use Case	Viteză
RecursiveCharacter	Paragrafe → linii → cuvinte	Default — 90% din cazuri	Medie
CharacterTextSplitter	Un singur separator	Texte simple, uniforme	Rapidă
TokenTextSplitter	Tokens (nu caractere)	Control precis tokens, embeddings	Lentă
✔ Recomandare: Începe cu RecursiveCharacterTextSplitter — funcționează bine în 90% din cazuri			

Chunk Size — Cum Alegi?

Mic

200–500 chars

- ✓ Precis, focus, rapid
- ✗ Context limitat, pierde big picture

Mediu

500–2,000 chars

- ✓ Balance optimal ✓

Mare

2,000–5,000 chars

- ✓ Context bogat
- ✗ Lent, costisitor, depășești limite

Reguli practice per tip de document:

Extracție date (facturi): `chunk_size=500` # Informație densă, precis

Analiză contracte: `chunk_size=1500` # Clauze pot fi lungi

RAG / search: `chunk_size=1000` # Balance retrieval + context

Advanced: LLM-based Chunking

Preview L4 — RAG

Aspect	Traditional	LLM-based
Cost upfront	✗ Ieftin	✓ Scump (LLM Calls)
Precizie	⚠ Mecanic (la X chars)	✓ Semantic (înțelege context)
Summaries	✗ Nu generează	✓ Auto-generate per chunk
RAG retrieval	⚠ Aproximativ	✓ Precis (reduce LLM calls)
Cost overall	✗ Mai scump cu RAG	✓ Mai ieftin overall!

Plătești mai mult upfront, dar economisești în RAG: retrieval precis + reranking eficient cu summaries

LLM-based Chunking — Cod

Traditional (mecanic)

```
1 # — Traditional (mecanic) —
2 splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
3 chunks = splitter.split_documents(docs)
4 # → Chunks la 1000 caractere
5 # → Fără summaries
```

LLM-based (semantic)

```
1 prompt = ChatPromptTemplate.from_messages([
2     ("system", ""Împarte textul în chunks semantice.
3     Pentru fiecare chunk, generează un summary.
4     Returnează JSON: {{chunks: [{{text, summary}}]}}"""),
5     ("user", "{document}"),
6 ])
7 result = llm.invoke(prompt.format_messages(document=doc))
```

Când folosești LLM chunking:

Facturi simple

✗ Traditional

Contracte complexe

✓ LLM-based

RAG cu retrieval frecvent

✓ LLM-based

Prototipare rapidă

✗ Traditional

Exemplu Practic: Factură PDF

```
1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3
4 # 1. Încarcă factura
5 loader = PyPDFLoader("factura_001.pdf")
6 documents = loader.load()
7
8 # 2. Configurează splitter pentru text dens (facturi)
9 splitter = RecursiveCharacterTextSplitter(
10     chunk_size=500,      # facturi: info densă
11     chunk_overlap=50,    # 10% overlap
12 )
13
14 # 3. Sparge documentele în chunks
15 chunks = splitter.split_documents(documents)
16
17 print(f"Pagini: {len(documents)}")
18 print(f"Chunks: {len(chunks)}")
19 print("--- Chunk 1 ---")
20 print(chunks[0].page_content)
21
```

Output →

 Pagini: 2  Chunks: 8

--- Chunk 1 ---

FACTURĂ NR. 2024-001, Data: 15.02.2024...

Metadata în Chunks

```
1  for i, chunk in enumerate(chunks):
2      chunk.metadata.update({
3          # Filtrare în RAG
4          "doc_type": "factura",
5          "client_id": "ACME-123",
6          "language": "ro",
7          "created_at": "2024-02-15",
8          # Debugging
9          "chunk_id": i,
10         # Citări (source & page vin deja de la loader)
11         "title": "Factură 2024-001",
12         # Deduplicare
13         "file_hash": file_hash,
14     })
15
```

Tracking

Știi de unde vine fiecare chunk
→ pagina + fișier sursă

Debugging

Găsești rapid sursa problemei
→ chunk 3/8 din pagina 2

RAG

Citări precise în răspunsuri
→ Lecția 4

Când NU Ai Nevoie de Chunking

1

Document mic

< 2,000 caractere

```
# Direct la LLM, no splitting  
llm.invoke(document.page_content)
```

2

Model cu context mare

Claude 3 (200K), Gemini (1M+)

```
# Trimite tot documentul odată  
# (atenție la cost!)
```

3

Procesare per pagină

Facturi 1-2 pagini

```
for page_doc in documents:  
    extract(page_doc)
```

Regula: Chunking doar când documentul nu încapă confortabil în context window

Sumar — Text Splitters & Chunking

`chunk_size`

500–2,000 caractere — dimensiunea unui chunk

`chunk_overlap`

10–20% din `chunk_size` — evita context rupt

Strategii:

Default (90%)

→ `RecursiveCharacterTextSplitter`

Control precis tokens

→ `TokenTextSplitter`

Precizie maximă + RAG

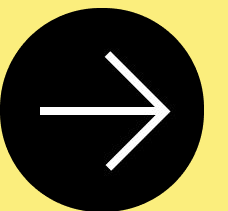
→ `LLM-based chunking (advanced)`

Best practice: Începe cu `chunk_size=1000`, `overlap=100` · Ajustează per tip document · Testează fragmentarea

Next → Secțiunea 3: Structured Extraction cu Pydantic

Extracție Structurată cu Pydantic

Extragerea datelor structurat din documente cu LLM



Problema: Text Nestructurat

Ce avem după loading + chunking:

```
1 text = ""
2 FACTURĂ NR. 2024-001
3 Data: 15.02.2024
4 Client: SC ClientCorp SA
5 Total: 9,520 RON
6 ""
```



Ce vrem:

```
1 expected = {
2     "numar": "2024-001",
3     "data": "2024-02-15",
4     "client": "SC ClientCorp SA",
5     "total": 9520.0,
6 }
```



Problema: LLM-ul returnează text liber, nu JSON consistent!

Soluția:

Structured Output cu Pydantic

Text brut

LLM + Schema

Pydantic object

JSON file



Ce Este Pydantic?

Pydantic = librărie Python pentru validare date

```
1 from pydantic import BaseModel
2
3 class Invoice(BaseModel):
4     numar: str
5     data: str
6     client: str
7     total: float
8
```

Beneficii:



Type safety

Validare automată a tipurilor



Default values

Câmpuri opționale cu fallback



Field descriptions

Documentație inline pentru LLM

LangChain + Pydantic = Structured Output garantat

Definirea Schemei Pydantic

```
1 from pydantic import BaseModel, Field
2 from typing import List
3
4 class Invoice(BaseModel):
5     """Schemă pentru extracție date factură."""
6
7     numar: str = Field(description="Numărul facturii (ex: 2024-001)")
8     data: str = Field(description="Data emiterii (format: YYYY-MM-DD)")
9     client: str = Field(description="Numele clientului")
10    furnizor: str = Field(description="Numele furnizorului")
11    total: float = Field(description="Suma totală în RON")
12
13    produse: List[str] = Field(
14        default=[], # câmp opțional
15        description="Lista produse/servicii", # ghidează LLM-ul
16    )
17
```

Docstring → context pentru LLM

Field(description=...) ghidează LLM-ul ce anume să extragă

List[str] → LLM înțelege că poate fi mai multe valori

default=[] → câmp opțional (dacă lipsește, nu eșuează)

Descrierile din Field() ghidează LLM-ul ce să extragă — cu cât mai precise, cu atât mai bune rezultatele!

LLM API Structured Output

API pot forța output-ul să urmeze schema Pydantic:

```
1 class Invoice(BaseModel):
2     numar: str
3     total: float
4
5 client = genai.Client()
6
7 # Forțăm output-ul să respecte schema Invoice
8 response = client.models.generate_content(
9     model="gemini-2.0-flash",
10    contents="Extrage datele din: FACTURĂ NR. 2024-001, Total: 9520 RON",
11    config={
12        "response_mime_type": "application/json",
13        "response_schema": Invoice,
14    },
15 )
16
17 invoice: Invoice = response.parsed
18 print(invoice.numar)    # "2024-001"
19 print(invoice.total)   # 9520.0
20 print(type(invoice))   # <class 'Invoice'>
```

Input

Text liber (string)

Magie

response_schema
face bridging-ul automat

Output

Obiect Pydantic — nu string!
Acces direct: invoice.total

Output = obiect Pydantic validat, nu string! → accesezi câmpuri direct: invoice.numar, invoice.total

Extraction Workflow

```
1 import json
2 from google import genai
3 from pydantic import BaseModel
4
5 # 1. Definește schema
6 class Invoice(BaseModel):
7     numar: str
8     total: float
9
10 # 2. Configurează clientul Gemini
11 client = genai.Client()
12
13 # 3. Extrage din text
14 invoice_text = "FACTURĂ NR. 2024-001\nTotal: 9,520 RON"
15 response = client.models.generate_content(
16     model="gemini-2.0-flash",
17     contents=invoice_text,
18     config={
19         "response_mime_type": "application/json",
20         "response_schema": Invoice,
21     },
22 )
23 result: Invoice = response.parsed
24
25 # 4. Salvează JSON
26 with open("factura_001.json", "w", encoding="utf-8") as f:
27     json.dump(result.model_dump(), f, indent=2, ensure_ascii=False)
28
```

Pipeline:



Prompt pentru Extracție



```
1 system_prompt = """Ești un expert în extragerea datelor din facturi.  
2 Extrage cu atenție toate câmpurile cerute.  
3 Dacă un câmp lipsește, lasă-l null."""  
4  
5 # User prompt cu placeholder – refolosit pentru orice factură  
6 def extract_invoice(invoice_text: str) -> Invoice:  
7     response = client.models.generate_content(  
8         model="gemini-2.0-flash",  
9         contents=f"Extrage datele din această factură:\n\n{invoice_text}",  
10        config={  
11            "system_instruction": system_prompt,  
12            "response_mime_type": "application/json",  
13            "response_schema": Invoice,  
14        },  
15    )  
16    return response.parsed  
17  
18 result = extract_invoice(text)  
19
```

De ce contează prompt-ul?

System prompt

Context + rol pentru LLM
→ rezultate mai consistente

Instrucțiuni clare

"Dacă lipsește → null"
→ evită halucinații

User prompt

Variabilă {invoice_text}
→ refolosești chain-ul

System prompt + Field descriptions + User prompt = triada câștigătoare pentru extracție precisă

Nested Schemas — Structuri Complexe



```
1  from pydantic import BaseModel, Field
2  from typing import List
3
4  class Product(BaseModel):
5      """Produs din factură."""
6      denumire: str
7      cantitate: int
8      pret_unitar: float
9      total: float
10
11  class Invoice(BaseModel):
12      """Factură completă."""
13      numar: str
14      data: str
15      client: str
16      # Listă imbricată – LLM-ul populează automat structura
17      produse: List[Product] = Field(default=[])
18      total_general: float
19
```

Structura imbricată:

Invoice

numar: str

data: str

client: str

produse: List[Product]

denumire: str

cantitate: int

pret_unitar: float

total: float

LLM-ul populează automat structura imbricată — funcționează și cu 3-4 niveluri de nesting!

Validare și Error Handling

```
1 invoice = None
2 for attempt in range(3):
3     try:
4         response = client.models.generate_content(
5             model="gemini-2.5-flash",
6             contents=text,
7             config={
8                 "response_mime_type": "application/json",
9                 "response_schema": Invoice,
10            },
11        )
12        invoice = response.parsed
13        break # validarea a trecut → ieșim din loop
14    except ValidationError as e:
15        print(f"Încercare {attempt + 1} eșuată: {e}")
16    except Exception as e:
17        print(f"Eroare generală: {e}")
18        break # erori de rețea/API → nu mai retry-uim
19
20 if invoice:
21     with open(f"{invoice.numar}.json", "w", encoding="utf-8") as f:
22         json.dump(invoice.model_dump(), f, indent=2, ensure_ascii=False)
23 else:
24     print("Toate retry-urile au eșuat")
25
```

Ce prinde fiecare except:

ValidationError

LLM a returnat câmpuri greșite sau lipsă

Exception

Erori de rețea, API, sau format neașteptat



Pydantic garantează că dacă nu aruncă excepție → datele sunt corecte și type-safe!

Exemplu Schema: Contract

Schema se adaptează la
tipul documentului:

Factură

numar, client, produse, total

Contract

prestator, valoare, durata,
obligatii

Raport

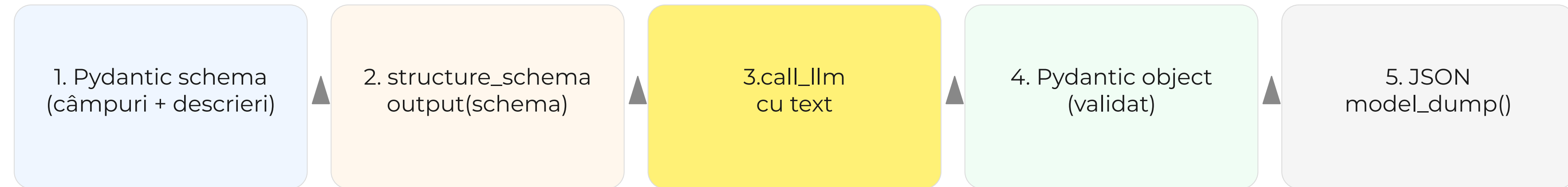
titlu, data, metrics, concluzii

```
1 class Contract(BaseModel):
2     """Schemă extracție date contract."""
3
4     numar: str = Field(description="Număr contract")
5     data_incheiere: str = Field(description="Data semnării")
6     prestator: str = Field(description="Numele prestatorului")
7     beneficiar: str = Field(description="Numele beneficiarului")
8     valoare: float = Field(description="Valoarea în RON")
9     durata_luni: int = Field(description="Durata în luni")
10
11     obligatii_prestator: List[str] = Field(
12         default=[],
13         description="Lista obligațiilor prestatorului",
14     )
15
```

Un LLM + o schemă diferită = extractor complet diferit → refolosești aceeași logică!

Sumar — Structured Extraction

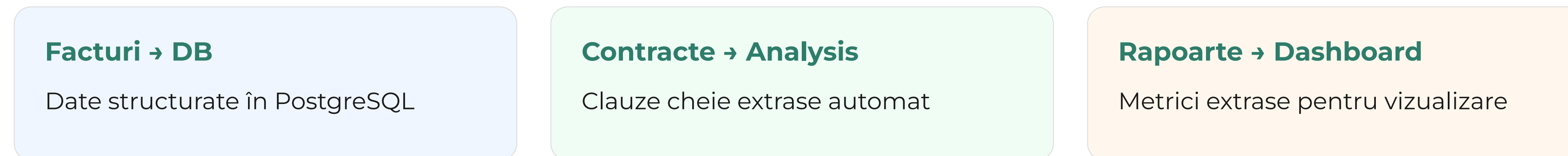
Workflow:



Avantaje:

- ✓ Output consistent (nu text liber)
- ✓ Validare automată Pydantic
- ✓ Type safety — câmpuri accesibile direct
- ✓ Ușor de salvat ca JSON

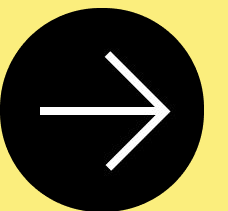
Use cases:



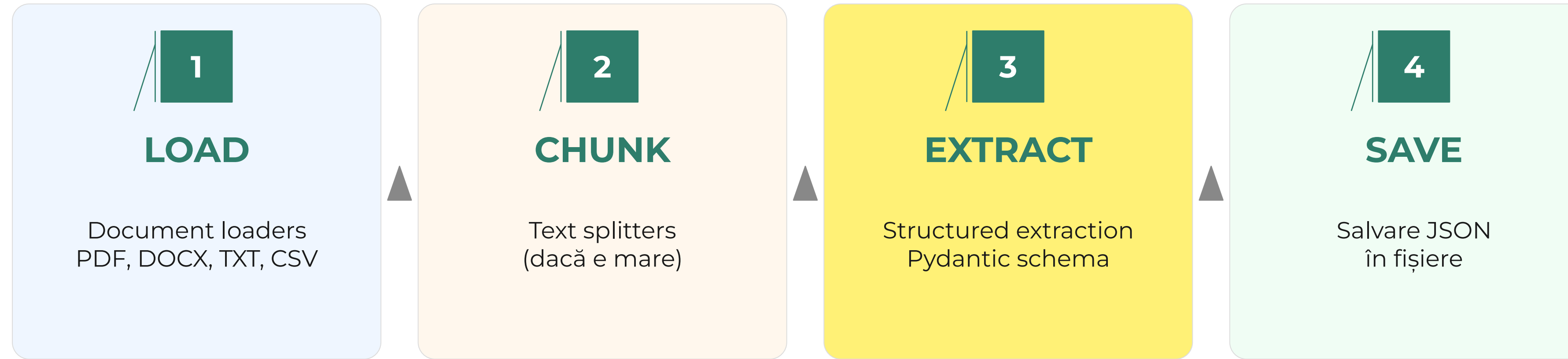
Next → Secțiunea 4: Building the Extraction Pipeline

Construirea Pipeline-ului

Load → Chunk → Extract → Save



Pipeline Overview



Input:

`factura.pdf` · `contract.docx` ·
`termeni.txt`

Output:

`factura.json` · `contract.json` · ...

Fiecare secțiune anterioară = un step din pipeline:

S1 — Document Loaders

→ **LOAD step**

S2 — Text Splitters

→ **CHUNK step**

S3 — Pydantic Schemas

→ **EXTRACT step**

Când Chunk-uim în Pipeline?

```
def should_chunk(document: Document, max_size: int = 4000) -> bool:
    """Decide dacă documentul trebuie chunk-uit."""
    return len(document.page_content) > max_size
```

Tip Document	Dimensiune tipică	Chunk?
Factură (1–2 pagini)	~2,000 chars	 Nu
Contract (5–10 pagini)	~8,000 chars	 Da
Raport (50+ pagini)	50,000+ chars	 Da

Regula: document < 4,000 caractere → procesează direct, fără chunking

Pipeline: Document Mic (Factură)

No chunking needed

```
1
2 def extract_invoice_simple(file_path: str) -> Invoice:
3     # 1. Load
4     docs = PyPDFLoader(file_path).load()
5
6     # 2. Combine pages (dacă e multi-page)
7     full_text = "\n\n".join(doc.page_content for doc in docs)
8
9     # 3. Extract direct (fără chunking)
10    response = client.models.generate_content(
11        model="gemini-2.0-flash",
12        contents=full_text,
13        config={
14            "response_mime_type": "application/json",
15            "response_schema": Invoice,
16        },
17    )
18    invoice: Invoice = response.parsed
19
20    # 4. Save
21    save_json(invoice, "extracted_data/")
22    return invoice
23
```

4 linii de logică:

1 Load
PyPDFLoader → lista de docs

2 Combine
Unești paginile cu \n\n

3 Extract
with_structured_output(Invoice)

4 Save
JSON pe disc

Pipeline: Document Mare (Contract)

Cu chunking

```
1 def extract_contract_chunked(file_path: str) -> Contract:
2     # 1. Load
3     docs = Docx2txtLoader(file_path).load()
4
5     # 2. Chunk (document mare)
6     splitter = RecursiveCharacterTextSplitter(
7         chunk_size=2000, chunk_overlap=200,
8     )
9     chunks = splitter.split_documents(docs)
10
11    # 3. Combine primele chunks (info cheie e la început)
12    text = "\n\n".join(c.page_content for c in chunks[:3])
13
14    # 4. Extract dintr-un singur call
15    response = client.models.generate_content(
16        model="gemini-2.0-flash",
17        contents=text,
18        config={
19            "response_mime_type": "application/json",
20            "response_schema": Contract,
21        },
22    )
```

Diferența cheie:

 **Document mic**

Tot textul → LLM direct

 **Document mare**

Chunk-uit → combine primele chunks → extract

Concatenezi primele 2-3 chunks (info cheie e la început) și extragi dintr-un singur call

Routing pe Tip Document

```
1 EXTRACTION_REGISTRY = {
2     "factura": {
3         "schema": Invoice,
4         "chunk_size": None,      # Nu chunk-ui facturi
5         "prompt": invoice_prompt,
6     },
7     "contract": {
8         "schema": Contract,
9         "chunk_size": 2000,
10        "prompt": contract_prompt,
11    },
12 }
13
14 def extract_document(file_path: str, doc_type: str):
15     config = EXTRACTION_REGISTRY[doc_type]
16     docs = load_document(file_path)
17
18     # Chunk doar dacă e necesar
19     if config["chunk_size"]:
20         docs = chunk_documents(docs, config["chunk_size"])
21
22     return extract_with_schema(docs, config["schema"])
23
```

Registry pattern:

factura

schema=Invoice
chunk_size=None

contract

schema=Contract
chunk_size=2000

Adaugi un tip nou?

Un singur entry în REGISTRY
→ zero modificări la logică

Error Handling în Pipeline



```
1 def extract_with_error_handling(file_path: str, doc_type: str):
2     # 1. Load
3     try:
4         docs = load_document(file_path)
5     except FileNotFoundError:
6         return {"error": "Fișier inexistent"}
7     except ValueError as e:
8         return {"error": f"Tip fișier nesuportat: {e}"}
9
10    # 2. Extract
11    try:
12        result = extract_with_schema(docs, doc_type)
13    except ValidationError as e:
14        return {"error": f"Validare eșuată: {e}"}
15    except Exception as e:
16        return {"error": f"Extracție eșuată: {e}"}
17
18    save_json(result, output_dir)
19    return {"success": True, "data": result}
20
```

Ce prinde fiecare:

FileNotFoundError

Fișierul nu există
sau calea e greșită

ValueError

Format nesuportat
(.xlsx, .pptx etc.)

ValidationError

LLM n-a extras
corect câmpurile

Exception

Network, API timeout
sau altceva

Returnează mereu: {"success": True, ...} sau {"error": "..."} — nu arunci excepții negestionate!

Salvare Organizată

Structura output:

```
└─ extracted_data/
   └─ facturi/
      ├── 2024-001.json
      ├── 2024-002.json
      └── 2024-003.json
   └─ contracte/
      ├── 123_2024.json
      ├── 124_2024.json
      └─ batch_results.json
```

`mkdir(parents=True, exist_ok=True)` → crează automat directoarele lipsă, fără eroare dacă există deja

Salvare Eficienta

```
1  STOP = None # sentinel propagat între workeri
2
3  def save_json(data: BaseModel, doc_type: str):
4      out = Path("extracted_data") / doc_type
5      out.mkdir(parents=True, exist_ok=True)
6      filename = f"{data.numar.replace('/', '_')}.json"
7      with open(out / filename, "w", encoding="utf-8") as f:
8          json.dump(data.model_dump(), f, indent=2, ensure_ascii=False)
9
10 def run_save_pipeline(files_to_process, num_workers: int = 5):
11     """Pornește workeri, procesează fișiere, oprește curat la final."""
12     queue: Queue = Queue(maxsize=100) # queue local, nu global
13
14     def worker(worker_id: int):
15         while True:
16             item = queue.get()
17             if item is STOP:
18                 queue.put(STOP) # propagă oprirea în cascadă
19                 break
20             data, doc_type = item
21             try:
22                 save_json(data, doc_type)
23                 log_to_db(data, doc_type)
24                 notify(data, doc_type)
25             except Exception as e:
26                 print(f"Worker {worker_id} error: {e}")
27
28     workers = [Thread(target=worker, args=(i,)) for i in range(num_workers)]
29     for w in workers: w.start()
30
31     # Producer
32     for file_path, doc_type in files_to_process:
33         result = extract_document(file_path, doc_type)
34         queue.put((result, doc_type))
35
36     # Semnal de final – un singur STOP, se propagă singur
37     queue.put(STOP)
38     for w in workers: w.join()
39
```

Batch Processing

```
1 # Helper la modul-level (cerință pentru pickle în multiprocessing)
2 def _process_one(file_path: str, schema: Type[BaseModel]):
3     # Pipeline reinstanțiat per worker → client Gemini propriu
4     pipeline = ExtractionPipeline()
5     return pipeline.process(file_path, schema)
6
7 def batch_extract(files: List[str], schema: Type[BaseModel],
8                  num_workers: int = 4):
9     results = {"success": [], "errors": []}
10
11     with ProcessPoolExecutor(max_workers=num_workers) as executor:
12         # Submit toate fișierele, păstrăm maparea future → file_path
13         future_to_file = {
14             executor.submit(_process_one, f, schema): f for f in files
15         }
16         # Procesăm rezultatele pe măsură ce sunt gata
17         for future in as_completed(future_to_file):
18             file_path = future_to_file[future]
19             try:
20                 data = future.result()
21                 results["success"].append({"file": file_path, "data": data})
22             except Exception as e:
23                 results["errors"].append({"file": file_path, "error": str(e)})
24
25     print(f"Success: {len(results['success'])}")
26     print(f"Errors: {len(results['errors'])}")
27     return results
28
```

Pattern batch:

Iterează fișiere

Un fișier pe rând
→ izolezi erorile

Try/except per fișier

Un fișier cu eroare
nu oprește batch-ul

Raportare rezultate

✓ success + ✗ errors
la final

Când folosești batch: 10+ documente · procesare nocturnă · import arhivă istorică

Complete Pipeline

```
1 class ExtractionPipeline:
2     """Pipeline complet de extracție – sigur de folosit în multiprocessing."""
3
4     def __init__(self, model: str = "gemini-2.0-flash"):
5         self.model = model
6         self.splitter = RecursiveCharacterTextSplitter(
7             chunk_size=2000, chunk_overlap=200,
8         )
9         self._client: genai.Client | None = None # lazy
10
11     @property
12     def client(self) -> genai.Client:
13         # Inițializat per proces – clientul nu e pickle-able
14         if self._client is None:
15             self._client = genai.Client()
16         return self._client
17
18     def process(self, file_path: str, schema: Type[BaseModel]):
19         # 1. Load
20         docs = load_document(file_path)
21         # 2. Chunk dacă e necesar
22         if self._should_chunk(docs):
23             chunks = self.splitter.split_documents(docs)
24             text = "\n".join(c.page_content for c in chunks[:3])
25         else:
26             text = "\n".join(d.page_content for d in docs)
27         # 3. Extract
28         responses = []
29         for chunk in chunks:
30             responses += [self.client.models.generate_content(
31                 model=self.model,
32                 contents=chunk,
33                 config={
34                     "response_mime_type": "application/json",
35                     "response_schema": schema,
36                 },
37             )]
38         return responses
39
40
```

De ce clasă?

`__init__`

Configurezi o singură dată:
llm + splitter

`process()`

Orice schemă, orice fișier
→ un singur entry point

`_should_chunk()`

Decizie internă
→ API public curat

Sumar — Extraction Pipeline

Load → Chunk → Extract → Save

Key Decisions:

Decision	Când Da	Când Nu
Chunk?	Document > 4K chars	Document mic (factură)
Multi-page?	PDF cu mai multe pagini	Single page
Batch?	10+ documente	1–2 documente

Best Practices:



Validează input înainte de procesare



Error handling la fiecare step



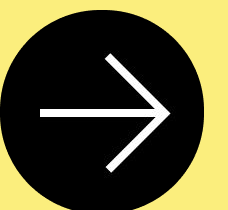
Salvează organizat (foldere per tip)



Log succese și erori pentru batch

Integrarea cu Chat Agent

Pipeline → Tool → Agent conversațional



De la Pipeline la Tool

Înainte (L1-2):

```
1 class ExtractInvoiceParams(BaseModel):
2     file_path: str = Field(description="Calea către factură", min_length=1)
3
4 @register_tool
5 def extract_invoice(params: ExtractInvoiceParams) -> dict:
6     """Extrage date structurate dintr-o factură."""
7     result = extract_invoice_data(params.file_path)
8     return result.model_dump()
```

Acum (L3):

```
1 # — Logică pură: extracția în sine (fără decorator) —
2 def extract_invoice_data(file_path: str) -> Invoice:
3     """Funcția de business – apelată și din pipeline, și din tool."""
4     pipeline = ExtractionPipeline()
5     return pipeline.process(file_path, Invoice)
```

Transformare:

ExtractionPipeline



extract_invoice (tool)

Pipeline reutilizabil · Interacțiune naturală · Agentul alege tool-ul automat

Multiple Tools pentru Tipuri Document

```
1 class ExtractInvoiceParams(BaseModel):
2     file_path: str = Field(description="Cale către factură", min_length=1)
3
4 class ExtractContractParams(BaseModel):
5     file_path: str = Field(description="Cale către contract", min_length=1)
6
7 @register_tool
8 def extract_invoice_tool(params: ExtractInvoiceParams) -> dict:
9     """Extrage date din FACTURI: numar, data, client, total, produse.
10     Folosește pentru facturi, invoices, documente financiare."""
11     pipeline = ExtractionPipeline()
12     return pipeline.process(params.file_path, Invoice).model_dump()
13
14 @register_tool
15 def extract_contract_tool(params: ExtractContractParams) -> dict:
16     """Extrage date din CONTRACTE: prestator, beneficiar, valoare, durată.
17     Folosește pentru contracte de prestări servicii,
18     acorduri comerciale, convenții."""
19     pipeline = ExtractionPipeline()
20     return pipeline.process(params.file_path, Contract).model_dump()
21
```

Cum alege agentul?

User: "Procesează factura X"

→ **extract_invoice_tool**

Bazat pe docstring!

User: "Extrage contractul Y"

→ **extract_contract_tool**

Bazat pe docstring!

Agentul citește descrierile (docstrings) și alege tool-ul potrivit automat — fără if/else manual!

Integrare cu Chat Agent



```
1 # 1. Tools – convertite din registry-ul propriu
2 tools = ToolWrapper.to_langchain_tools()
3 llm_with_tools = llm.bind_tools(tools)
4
5 # 2. System prompt – randat cu tools active
6 base_prompt = """Ești un asistent AI care procesează documente.
7 Poți extrage date din facturi și contracte.
8 Când userul cere procesarea unui document, folosește tool-ul potrivit."""
9 prompt = render_prompt_with_tools(base_prompt)
10
11 # 3. Observabilitate – callback LangFuse opțional
12 langfuse = get_langfuse_handler()
13 callbacks = [langfuse] if langfuse else []
14
15 # 4. Invoke
16 messages = [
17     SystemMessage(content=prompt),
18     HumanMessage(content="Extrage datele din sample_docs/factura_001.pdf"),
19 ]
20 response = llm_with_tools.invoke(messages, config={"callbacks": callbacks})
21
```

4 pași:

1

Tools list

Listă de tool-uri disponibile

2

System prompt

Context + instrucțiuni agent

3

Agent

create_tool_calling_agent()

4

Executor

AgentExecutor rulează loopul

User Interaction Flow

User: `"Procesează factura din sample_docs/factura_001.pdf"`

Agent: `Analizează cererea → "User vrea să extragă date dintr-o factură"`

Agent: `Alege tool: extract_invoice_tool
Parametri: file_path="sample_docs/factura_001.pdf"`

Tool: `Execută: Load → Extract → Save`

Tool: `Returnează: {numar: "2024-001", total: 9520, ...}`

Agent: `"Am procesat factura 2024-001. Total: 9,520 RON.
Salvat în extracted_data/facturi/2024-001.json"`

Conversație Naturală cu Agentul

Exemplu sesiune completă:

 **User:**

Am o factură în `sample_docs/factura_001.pdf`, procesează-o

 **Agent:**

[`extract_invoice_tool`] → ✓ Factură 2024-001 procesată. Total: 9,520 RON

 **User:**

Cât costă licența software?

 **Agent:**

[citește din rezultat anterior] → Licența Software Enterprise: 5,000 RON
(10 × 500 RON)

 **User:**

Ți contractul din `contract_lung.txt`?

 **Agent:**

[`extract_contract_tool`] → ✓ Contract 123/2024. Valoare: 50,000 RON, 12
luni

Notă: conversation memory (L8) necesară pentru context între invoke-uri separate

Error Handling în Tool



```
1 class ExtractInvoiceParams(BaseModel):
2     file_path: str = Field(description="Calea către factură", min_length=1)
3
4 @register_tool
5 def extract_invoice_tool(params: ExtractInvoiceParams) -> dict:
6     """Extrage date dintr-o factură."""
7     try:
8         invoice = extract_invoice_data(params.file_path)
9         return {
10             "success": True,
11             "data": invoice.model_dump(),
12             "message": f"Factură {invoice.numar} procesată",
13         }
14     except FileNotFoundError:
15         return {"success": False, "error": "Fișierul nu a fost găsit"}
16     except ValidationError as e:
17         return {"success": False, "error": f"Validare eșuată: {e}"}
18     except Exception as e:
19         # Niciodată exception raw – agentul nu poate citi traceback
20         return {"success": False, "error": f"Eroare procesare: {str(e)}"}
21
```

De ce returnăm dict, nu raise?

Agentul poate citi eroarea

și o poate comunica
user-ului în limbaj natural

Nu crăpă AgentExecutor

Loop-ul continuă chiar
dacă un tool eșuează

User primește feedback clar

"Fișierul nu a fost găsit"
not exception traceback

Regula: tool-ul returnează MEREU un dict — cu "success": True sau "success": False, niciodată exception raw

Sumar — Pipeline → Tool → Agent

User request → Agent decide tool → Tool execută pipeline → Agent răspunde

Ce am făcut în Secțiunea 5:

✓ 1. Wrapat ExtractionPipeline ca @tool

✓ 2. Definit descrieri clare (ghidează agentul)

✓ 3. Integrat cu AgentExecutor (din L1–2)

✓ 4. User interacționează natural cu agentul

Beneficii:

✓ Conversație naturală ("Procesează factura X")

✓ Agent alege tool-ul potrivit automat

✓ Context cu memory (L8)

✓ Pipeline reutilizabil pentru orice document

Next Steps: L4: PostgreSQL + RAG · L5: LangGraph workflows · L6: Multi-agent (Analyst + Extractor)

Lecția 03 — Completă! Document Processing Pipeline gata de extins în lecțiile următoare.